

# LECTURE-15

Tuesday (online)  
31/3/26.

□ Semaphores: It is simply a non-negative shared variable used to solve critical section problem to achieve process synchronization.

```
do {  
    Semaphore  
    wait(S) // Critical section  
    Signal(S) // Remainder section  
} while(1);  
  
wait(S) {  
    while (S <= 0) {  
        ; // No operations for waiting  
    }  
    S--;  
}  
  
Signal(S) {  
    S++;  
}
```

Working:

Semaphore,  $S$ , will start from 1.

When a process  $P_1$  is being executed & it comes to  $\text{wait}()$  with  $S=1$ .  
 $\text{wait}()$  func. will check if  $S=1$ , then that process will get Critical Section (CS) after decrementing 'S'.  $S$  is decremented first so its value goes to 0 from 1 &  $P_1$  gets CS. Now if  $P_2$  arrives & bumps into  $\text{wait}()$ ; it will be stuck in loop for waiting where there is "No operation;" as  $S \leq 0$ .  
When  $P_1$  is done using CS in main program (with pink); it will go in  $\text{signal}()$  and increment  $S$  to 1 from 0. Now the stuck  $P_2$  is free and can use CS.

Mutual exclusion: ✓

Progress: ✓

Bounded waiting: X Since no Queue so ↑ starvation

⇒ Busy waiting: Since we've ; inside while loop so waiting process keeps on checking if  $S$  has become 1; it just do this useless job so CPU is Busy waiting, which just wastes CPU cycles.

Solution: Use  $\text{Block}()$  inside  $\text{wait}()$  and  $\text{wakeup}()$  inside  $\text{signal}()$ .



To solve Busy waiting & Bounded waiting problem:

```
typedef struct {
    int value; // actual semaphore no.
    struct process * L;
} Semaphore;
```

name of  
struct

creates a  
Linked List L

```
void wait (Semaphore S) {
    if (S.value < 0) {
        // add Process in S.L;
```

Block();

} puts process on wait and  
avoid it from spinning again &  
again in while loop.

object of  
struct

```
void Signal (Semaphore S) {
    S.value++;
    if (S.value <= 0) {
```

// remove process from S.L

wakeup (process);

} release current process

} using CS so blocked  
processes can use it.

This list is to solve bounded waiting problem as  
it makes queue to keep track that which process went to block first &  
that shld've CS first so no starvation

Q: Ensure S1 executes after S2. And S2 executes after S3.

|   | P1       | P2         | P3         |                                                                                     |
|---|----------|------------|------------|-------------------------------------------------------------------------------------|
| ① | -        | -          | -          | Sol: These processes are running vertically<br>downwards simultaneously (row-wise). |
| ② | wait(S2) | wait(S1)   | -          | ∴ we need to set semaphore                                                          |
| ③ | S1       | S2         | S3         | S1 & S2 in a way they gets                                                          |
| ④ |          | Signal(S2) | Signal(S1) | executed like: S3 → S2 → S1.                                                        |

answer: S1 = 0, S2 = 0 (in beginning).

Explanation: As row ② gets executed; P1 & P2 goes to waiting state as their  
parameter semaphores (S1 & S2) are initially 0. Then row ③ gets executed  
only for P3 and S3 gets executed in that as there was No wait() for it.  
Then row ④ executes for P3 and finally releases P2 as signal(S1) increments  
S1 from 0 → 1 and wait(S1) is over for P2. Then row ② and ③  
gets executed for P2 and then at row ④ for P2; signal(S2) releases  
P1 as S2 is now incremented from 0 → 1. Then S1 executed.



Q: Thread T1 prints 'a' and T2 prints 'b'. Synchronize them using semaphores so pattern is abb...

Ans:  $S_1 = 0, S_2 = 0, \text{int counter} = 0$ .

```
void T1() {
    wait(S1);
    print('a');
    signal(S2);
}
```

```
void T2() {
    wait(S2);
    counter++;
    print('b');
    if (counter % 2 == 0) {
        signal(S1);
    } else {
        signal(S2);
    }
}
```

Problems with semaphores: Since there are <sup>a lot of</sup> wait() & signal() used in diff. processes; jumbling them up will cause serious "deadlocks" & "starvation" and "Mutual Exclusion".

E.g. deadlocks

$S = 0 = 1$

P1

P2

wait(S)

wait(Q)

wait(Q)

wait(S)

signal(S)

signal(Q)

signal(Q)

signal(S)

Allow both P1 & P2 to use

CS when  $S = 1$ .

Should be like this to solve deadlock (same order)

E.g. Starvation

wait(S)

wait(S)

wait(S)

signal(S);

shld be signal(S).

E.g. mutual Exclusion

P1

P2

signal(S)

wait(S)

wait(S)

signal(S)